

Decision Architecture Platform

A unified, design-time-to-run-time decision architecture for engineering programs — and for the AI agents that increasingly support them.

This platform combines two documents into one: the original Agile Decision Architecture (Parts I–III), and its Holonic Extension (Part IV), which closes the run-time gap the original left open and shows that the same structural fix — bounded local determinism, federated through an audited seam — is also the answer to how multi-agent AI systems stay capable and aligned at once.

CONTENTS

- | | |
|---|--|
| 1. The Decision Triad: One Architecture, Not Three Methods | 7. The Decision Architecture Across Program Phases |
| 2. Response Surface Methods: Compressing Exploration Cost | 8. The Knowledge Graph: Making Traceability Structural |
| 3. Pareto Optimality: Replacing Opinion with Structured Trade-Off | 9. Competitive Advantage |
| 4. MBSE Architecture: The Authoritative System Model | 10. From Centralized Model to Holarchy |
| 5. Physics Simulation: Modelica and Multi-Domain Co-Simulation | 11. Reachability Isn't Causality |
| 6. Controls Integration and Hardware Verification | 12. What This Resolves for Multi-Agent Alignment |
| | 13. Implementation Maturity, Extended |
| | 14. Conclusion |
| | 15. Appendix: Quick Reference |

Preface: The Real Constraint Is Not Compute — It Is Decision Latency

Every engineering program has a binding constraint. In modern aerospace, defense, pharmaceutical, and industrial programs, that constraint is rarely computing power, tooling capability, or engineering talent. It is the time between when a consequential question arises and when a confident, defensible answer is available to the people who need to act on it.

Consider what happens when the chief architect asks: "If we move the power module 15 centimeters forward, what happens to thermal margins, structural weight, and control system

stability?" The answer that should arrive in hours instead arrives in weeks — because the thermal, structures, and controls teams each need to rerun their analyses, reconcile their models, and resolve conflicting assumptions before any integrated answer is possible. This latency is not a symptom of insufficient tooling. It is a symptom of disconnected tooling — three capable simulations that do not share a common model of the system, do not communicate results in a form the others can consume, and do not connect their outputs to the formal requirements against which the decision must ultimately be judged.

The engineering programs that consistently deliver on schedule, within budget, and with fewer late-stage surprises are not the ones with the highest-fidelity individual simulations. They are the ones with the shortest cycle from engineering question to confident, integrated, requirements-traceable answer.

Parts I–III below lay out the original decision acceleration architecture: MBSE simulation, Response Surface Methods, and Pareto Optimality, unified through a knowledge graph and closed by a Circuit Breaker pattern for AI-assisted work. Part IV extends it with a holonic governance layer, closing the run-time gap the original left open.

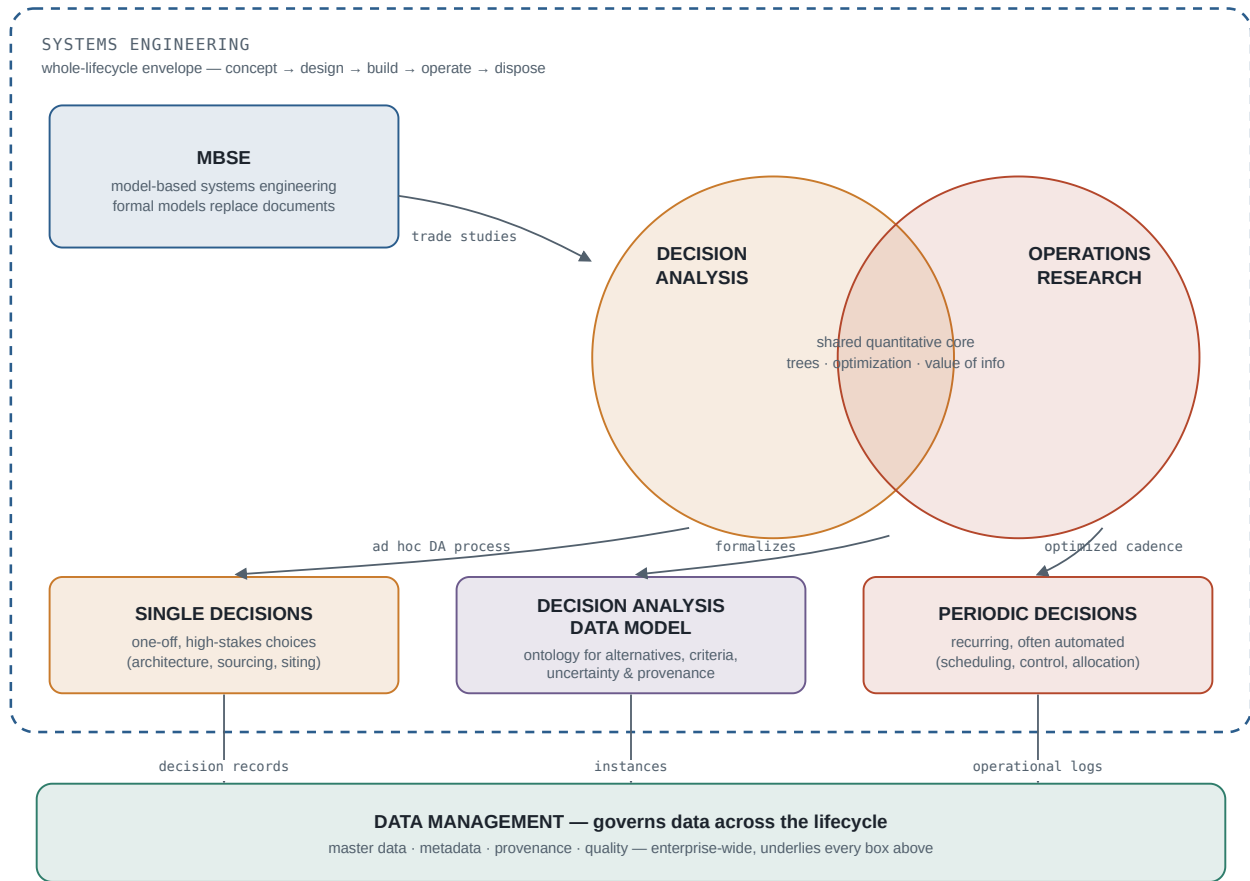


Figure 0 — the conceptual map. Systems engineering is the envelope; MBSE is how it's modeled. Decision analysis and operations research share a quantitative core that resolves into single and periodic decisions and one shared artifact, the decision analysis data model — all of it ultimately recorded and governed as data. Every method named in Parts I–IV below sits somewhere on this map.

The Decision Acceleration Architecture

Why three methods, used together, create a fundamentally different decision tempo

1. The Decision Triad: One Architecture, Not Three Methods

Decision outcomes in this section enable:

- Understand why each method is necessary but insufficient alone
- See the pipeline through which MBSE, RSM, and Pareto connect
- Identify which method addresses which class of engineering decision

1.1 The Question Each Method Answers

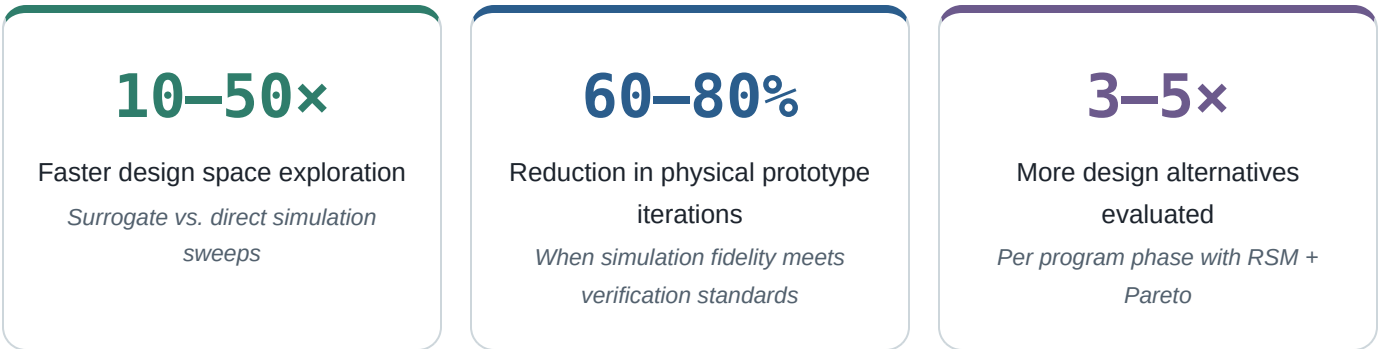
METHOD	THE QUESTION IT ANSWERS	WHAT IT PRODUCES	WHAT IT CANNOT DO ALONE
MBSE Simulation	What will this system actually do when all physics and behaviors interact?	Verified, requirements-traceable system behavior	Cannot efficiently sweep design space; too expensive per evaluation
Response Surface Methods	How does performance vary across my design space, and where is the optimum?	A cheap surrogate that can be evaluated thousands of times	Cannot verify requirements; cannot handle multi-objective trade-offs
Pareto Optimality	What trade-offs are genuinely unavoidable, and what is the efficient frontier?	A structured decision surface for stakeholder choice	Cannot evaluate system behavior; needs data from the other two

The critical insight is that these three methods form a pipeline, not a menu. MBSE simulation provides the high-fidelity physics ground truth. RSM samples are strategically simulated to build a surrogate that can be evaluated thousands of times at negligible cost. The surrogate is then swept by a Pareto optimizer to map the trade-off frontier. Stakeholders select from the

frontier with full awareness that it is grounded in validated physics — not in opinion, not in a single design point, and not in somebody's spreadsheet.

1.2 The Decision Latency Problem in Concrete Terms

Without the pipeline: a design change is proposed, each discipline reruns its analysis independently, results are reconciled in a review meeting, conflicts are resolved through negotiation rather than computation. This cycle takes weeks. With the pipeline: the surrogate is queried instantly, the Pareto frontier is available in the room, and the decision conversation focuses on which point on the frontier best serves program objectives — a conversation that happens in hours.



1.3 The Architecture in One Diagram

The pipeline flows in one direction for computation and in the reverse direction for requirements verification. This bidirectionality is what makes it a decision architecture rather than just a simulation workflow.

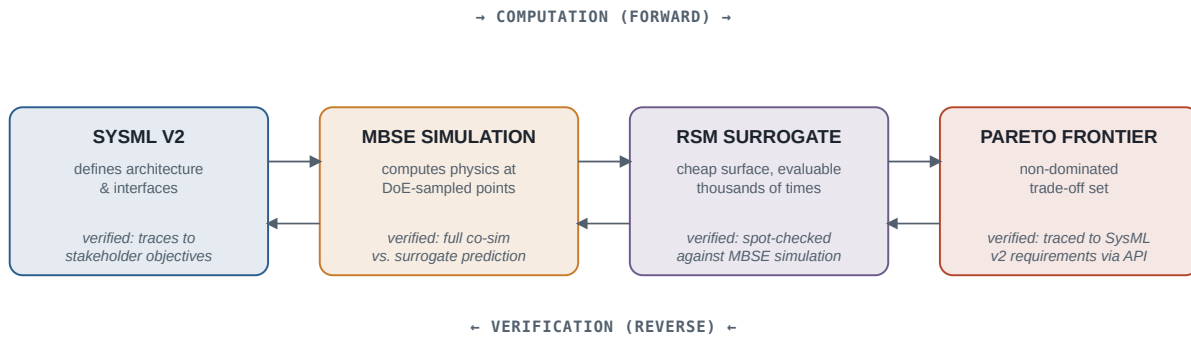


Figure 1 — the architecture in one diagram. Solid arrows: forward computation, SysML v2 through the Pareto frontier. Dashed arrows: reverse verification — every frontier candidate traces back to the requirement it satisfies.

The verification reverse-flow is what elevates this from a simulation workflow to a decision architecture. Every point on the Pareto frontier can be traced back to the formal requirements it satisfies. When a stakeholder selects a design, they are not choosing a simulation result — they are choosing a requirements-traceable system configuration.

2. Response Surface Methods: Compressing Exploration Cost

Decision outcomes in this section enable:

- Select the appropriate DoE design for a given decision context
- Understand when the surrogate is trustworthy and when it is not
- Connect RSM output directly to Pareto optimization input

The decision problem is straightforward: the system model is too expensive to evaluate at every point in the design space that would be needed to find the optimum by brute force, but design decisions must be made before you can afford to wait for exhaustive analysis. RSM resolves this tension by running a small, strategically chosen set of evaluations, fitting a mathematical surface to the results, and optimizing the surface instead of the system. The keyword is "strategically" — RSM uses Design of Experiments theory to choose evaluation points that maximize information about the response surface per unit of experimental or computational cost.

THE MATHEMATICAL SURFACE

RSM fits a second-order polynomial to the sampled data. For two design variables x_1 and x_2 : $\hat{y} = \beta_0 + \beta_1x_1 + \beta_2x_2 + \beta_{12}x_1x_2 + \beta_{11}x_1^2 + \beta_{22}x_2^2$. The six terms capture a constant baseline, two linear main effects, one two-way interaction, and two pure quadratic curvature terms, estimated by ordinary least squares. For k design variables the model has $(k+1)(k+2)/2$ terms — 6 for two variables, 10 for three, 15 for four. Beyond 6–7 variables, run counts to estimate all terms become prohibitive, and Kriging (Gaussian Process regression) or other machine-learning surrogates scale better dimensionally.

DESIGN SELECTION

DESIGN	RUN COUNT (K=3)	STRENGTHS	WHEN TO CHOOSE IT
Central Composite (CCD)	20	Estimates all quadratic terms; excellent prediction at design space boundaries	Default choice when boundary behavior matters; when all factor combinations are feasible
Box-Behnken	13	Avoids extreme corner combinations; efficient for 3–4 factors	When extreme factor combinations are infeasible, dangerous, or cost-prohibitive
Full factorial (reference)	125	Complete information; no assumptions	Computationally cheap responses; used as validation benchmark, not for surrogate building
Kriging + LHS	Flexible	Handles nonlinearity and discontinuities; scales to higher dimensions	When response is suspected to be highly nonlinear or when $k > 6$

VALIDATION – WHEN TO TRUST THE SURROGATE

A surrogate that has not been validated is a liability, not an asset. Three checks are required before using the surface for decision-making: **lack-of-fit test** (does the quadratic model adequately represent the data, or are higher-order terms present); **cross-validation** (leave-one-out — remove each design point, refit, predict the removed point); and **confirmation runs** (3–5 new evaluations at the predicted optimum and Pareto frontier candidates before committing to a design decision).

Critical boundary condition

A surrogate is only valid within the design space where it was fitted. Using it to predict behavior at factor values outside the original experimental region — extrapolation — produces results that may appear numerically reasonable while being physically meaningless. Every decision made from a surrogate must be explicitly checked against that boundary.

CONNECTING RSM TO PARETO

Once a validated surrogate exists for each system objective, the Pareto optimizer sweeps all surrogates simultaneously, evaluating thousands of design configurations in seconds. The decision team selects candidate designs — typically 3–5 points representing different trade-off positions — for verification against the full MBSE simulation. If verification matches surrogate predictions within tolerance, the decision is made; if not, the surrogate is refined in the region of unacceptable error. This verify-then-decide workflow is what separates responsible use of surrogates from naive use: the surrogate accelerates exploration, the full simulation verifies the decision.

3. Pareto Optimality: Replacing Opinion with Structured Trade-Off

Decision outcomes in this section enable:

- Replace unstructured design review debates with Pareto-grounded trade-off conversations
- Select from the frontier using methods appropriate to the decision context
- Identify knee points and non-convex regions that reveal design paradigm boundaries

Most engineering decisions involve multiple objectives that are genuinely in conflict. In unstructured design reviews these conflicts are resolved through negotiation, seniority, or organizational inertia — not through explicit reasoning about what is physically possible. Pareto analysis replaces that negotiation with a structured decision surface: the frontier. Every point on it cannot be improved on any objective without degrading another; every point below it is dominated, and choosing a dominated design is never rational.

The Pareto frontier does not make the decision. It eliminates the decisions that do not need to be made — all dominated designs — and structures the one conversation that does need to happen: which combination of trade-offs best serves program objectives? This is a fundamentally different conversation from "which design does the lead engineer prefer?"

Solution A dominates B if A is at least as good as B on every objective and strictly better on at least one — a relationship determined entirely by objective function values, requiring no weights or preferences. For discrete sets, frontier construction tests every pair; for continuous spaces, Evolutionary Multi-objective Optimization (NSGA-II for two–three objectives, NSGA-III for four or more) evolves a population toward the frontier using Pareto rank and crowding distance.

READING THE FRONTIER

FRONTIER SHAPE	WHAT IT SIGNALS	DECISION IMPLICATION
Smooth convex curve	Trade-offs are gradual and consistent	Weighted sum optimization reliably finds the correct solution; weights can be specified a priori
Strongly curved (knee)	One region offers high simultaneous value on all objectives	The knee point is the rational default when stakeholder preferences are unclear
Non-convex region	Some trade-off combinations require designs weighted sums cannot reach	Reference point or epsilon-constraint methods are required for complete coverage
Discontinuous (gap)	Two distinct design paradigms exist, no smooth transition	Programs choosing hybrid designs may be selecting provably inferior solutions
Flat along one axis	One objective is insensitive to design choices in this region	More design freedom exists than expected; relaxing that constraint enables optimization elsewhere

SELECTING FROM THE FRONTIER

Weighted sum scalarization assigns weights before seeing the frontier — works when preferences are stable and the frontier is expected to be convex. **Reference point methods** (NSGA-III, epsilon-constraint) let the team specify aspiration levels ("at least 85% reliability, at most \$2M unit cost") and find the closest frontier point — the standard approach under formal requirements. **Knee point selection** identifies maximum curvature — the rational choice when stakeholders cannot articulate preferences before seeing the frontier.

Hypervolume quantifies frontier quality as a single scalar, used to benchmark optimization runs and verify convergence.

The MBSE Simulation Engine

The technical infrastructure that grounds surrogate models and decision evidence in validated physics

4. MBSE Architecture: The Authoritative System Model

Decision outcomes in this section enable:

- Use SysML v2 to make control requirements formally verifiable rather than documentarily described
- Connect the architecture model to simulation through typed interfaces and the REST API
- Understand when to route to SysML vs. AADL for different architectural concerns

The architecture model is the system's authoritative semantic contract — the document that says what the system is, what interfaces it presents, what requirements it must satisfy, and how behavior is allocated across physical components. When that contract is formal enough to be queried programmatically, every downstream simulation, verification test, and Pareto frontier candidate can be checked against it automatically. Without a formal architecture model, connections between design decisions and requirements are maintained in documents and spreadsheets; changes propagate through email; conflicts surface in program reviews. With a formal architecture model accessible through a standard API, those connections are structural — the cost of each decision is a query, not a review cycle.

SYSML V2 FEATURE	TECHNICAL MECHANISM	DECISION VELOCITY IMPACT
Normative textual syntax	Models are text, stored in Git, diffed line-by-line	Architecture changes are trackable, reviewable, auditable at commit granularity
REST API repository	Standard HTTP endpoints for element CRUD and query	Any tool can query the architecture model without vendor adapters
Formal constraint requirements	RequirementUsage carries machine-evaluable constraint expressions	Simulation results checked against requirements automatically — no manual comparison
Strong typing with ISO 80000 units	Port declarations carry physical quantity kinds and units	Interface mismatches (mass vs. force, °C vs. °F) caught at model assembly, not test
Occurrence semantics	Elements have temporal life histories natively	Structural model connects directly to time-domain simulation without translation

AADL (SAE AS5506) is the right tool when the primary question is "will this software execute correctly on this hardware within these timing constraints?" — it generates mathematical proof of schedulability, a certificate no equivalent exists for in SysML. For avionics under DO-178C, automotive under ISO 26262, and safety-critical embedded systems requiring formal timing verification, AADL is not optional. The mature integration pattern uses both: SysML v2 at the system level for function allocation and interfaces, AADL at the software level for scheduling and safety proof, with traceability connecting them.

The REST API makes the architecture model a live participant in the decision pipeline: `GET /projects/{id}/commits/{commitId}/elements` retrieves all model elements at a specific architecture version; `POST /projects/{id}/commits` commits architecture changes, versioned and auditable; `POST /projects/{id}/commits/{commitId}/query-results` filters elements for automated checks. A simulation configuration validated against architecture commit `abc123` is explicitly tied to that version — when the architecture changes, the system knows which simulations and verification results are now stale structurally, not because someone remembered to check.

5. Physics Simulation: Modelica and Multi-Domain Co-Simulation

The decision problems that matter most in systems engineering are not single-domain problems — they are problems where the coupling between domains determines the answer. Modelica's acausal, equation-based architecture means a modeler writes physical laws, not

signal flow diagrams; the same component model works in any system context because causality is resolved at system assembly time by the equations, not fixed in advance.

SYSTEM TYPE	MODELICA STRENGTH	LIMITATION	SUPPLEMENT WITH
Multi-domain physical (thermal-mechanical-electrical)	Native strength; acausal coupling handles cross-domain interactions automatically	None significant for lumped-parameter systems	—
Automotive powertrain + BMS + controls	Industry standard; DLR, BMW, Toyota validated libraries	High-frequency switching power electronics	SPICE tools for switching circuits; ROM export via FMI
Building energy and HVAC	Lawrence Berkeley LBNL Buildings library; DOE validated	Full 3D airflow modeling	CFD offline for HTC correlations; feed into Modelica
Aerospace ECS and hydraulics	ThermoSysPro aerospace libraries; ESA qualified	Spatially distributed structural behavior	ANSYS/Nastran ROM exported as FMU
High-frequency power electronics	Poor fit; solver restarts at every switching event degrade performance	Fundamental	LTspice, Cadence for detailed design; behavioral ROM for system level

The Functional Mockup Interface (FMI) standard lets Modelica thermal models, Simulink control models, ANSYS structural ROMs, and AADL software timing models operate simultaneously in a co-simulation without any tool understanding another's internals — an FMU declares every variable it exposes (name, type, unit, causality) in a model description any conformant host can read. Three coordination schemes: **Jacobi (parallel)** — efficient, one-step delay, weakly coupled systems; **Gauss-Seidel (sequential)** — reduces delay error for asymmetric coupling; **Iterative (Newton-Raphson)** — near-monolithic accuracy at $k\times$ cost, required for strongly coupled bidirectional systems.

The communication step size trade-off

The communication step size h between FMU exchanges must be at least $10\times$ smaller than the fastest dynamic time constant at any interface. A 1-second thermal-mechanical coupling tolerates $h = 100\text{ms}$; a 1ms electrical transient requires $h = 0.1\text{ms}$ or domain decoupling. Violating this rule produces trajectories that are numerically smooth but physically wrong — the most dangerous simulation failure mode.

Thermal simulation occupies a unique position because heat is the medium through which most other domains interact — simultaneously a receiver from and a provider to every other

physics domain. One-way coupling (export temperature as a structural load) is adequate when deformations don't change thermal boundary conditions; two-way coupling through FMI is required when they do. MBSE simulation outputs become RSM inputs through a structured sampling protocol: the DoE design specifies evaluation points, each run produces the objective values the RSM surface is fit to — forcing the team to specify in advance what design variables are being studied and what objectives are being measured, surfacing disagreements explicitly rather than implicitly.

6. Controls Integration and Hardware Verification: Closing the Loop

The systems architect's control interface declaration and the controls engineer's Simulink model occupy different worlds until a formal, versioned, automatically checked connection exists between them — SysML v2's formally typed ports, carrying ISO 80000 quantity kinds, combined with the REST API, provide the technical foundation. The control law is only as good as the plant model it was designed against; the appropriate response is to trace which plant model fidelity level validated each control requirement, so the system knows which requirements need re-verification when the plant model updates.

STAGE	REAL HARDWARE?	WHAT IT PROVES	WHAT IT CANNOT PROVE	CERTIFICATION ROLE
MiL	None	Control law behavior against idealized plant under floating-point arithmetic	Code generation artifacts; real-time timing; I/O electrical characteristics	Control law requirements; initial stability margin verification
SiL	Compiled code on PC	Fixed-point arithmetic; quantization effects; code generation correctness	Processor timing; bus loading; I/O hardware behavior	Software unit and integration test evidence
PiL	Real processor only	Interrupt latency; RTOS scheduling; processor-specific cache effects	I/O electrical characteristics; full bus loading under realistic traffic	Timing requirement evidence on target hardware
HiL	Full ECU + RT sim	Closed-loop system with all hardware timing; real I/O; failure mode response	Environmental stress; vibration; full system integration	Subsystem validation evidence; DO-178C / ISO 26262 compliance

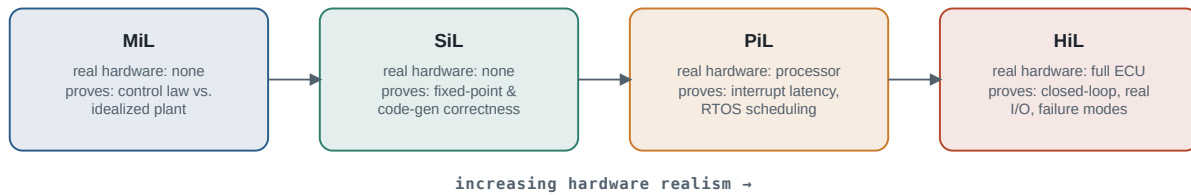


Figure 2 — the verification progression. Each stage reveals a class of failure mode the previous one structurally cannot: HiL is the program's last inexpensive opportunity to find failures that would be expensive, dangerous, or impossible to find in physical system integration.

Programs that truncate HiL coverage to save schedule are typically trading a short-term schedule recovery for a late-phase integration risk whose resolution cost is 10 to 100 times larger.

HiL test results are not just verification artifacts — they are decision inputs. A Pareto frontier candidate that has passed HiL verification under all relevant operating conditions and failure modes carries qualitatively higher decision confidence than one validated only against a MiL surrogate. Making this confidence distinction explicit, traceable, and queryable requires the knowledge graph infrastructure described next.

Operationalizing the Decision Architecture

Program-phase implementation, knowledge graph integration, AI grounding, and competitive outcomes

7. The Decision Architecture Across Program Phases

PROGRAM PHASE	DOMINANT DECISION	PRIMARY METHOD	SUPPORTING METHODS	OUTPUT
Concept and trade studies	Which system concept is architecturally viable?	Pareto + RSM (lightweight surrogates)	SysML v2 for interface definition	Frontier over concept alternatives; dominated concepts eliminated
Preliminary design	What are the key design parameters and their sensitivities?	RSM (DoE sampling of MBSE simulation)	Pareto for trade-off characterization	Validated surrogate; ranked sensitivity; design space map
Critical design	Does the selected design meet all requirements at all operating conditions?	MBSE simulation (full co-simulation)	SysML v2 requirements traceability	Verification evidence matrix traceable to architecture requirements
Integration and test	Are the real hardware and software performing as the model predicted?	HiL (model vs. hardware correlation)	MBSE model correlation update	Model update with correlated hardware behavior; residual risk characterization
Operate and sustain (v2)	Is the deployed system still inside its validated envelope, and what should change?	Digital-twin MBSE + periodic RSM/OR re-optimization against live telemetry	Domain events trigger re-verification; recursive Circuit Breaker validates AI-assisted operational recommendations	Updated Pareto frontier; re-verified requirements; field-correlated model; MTBH-tracked AI grounding

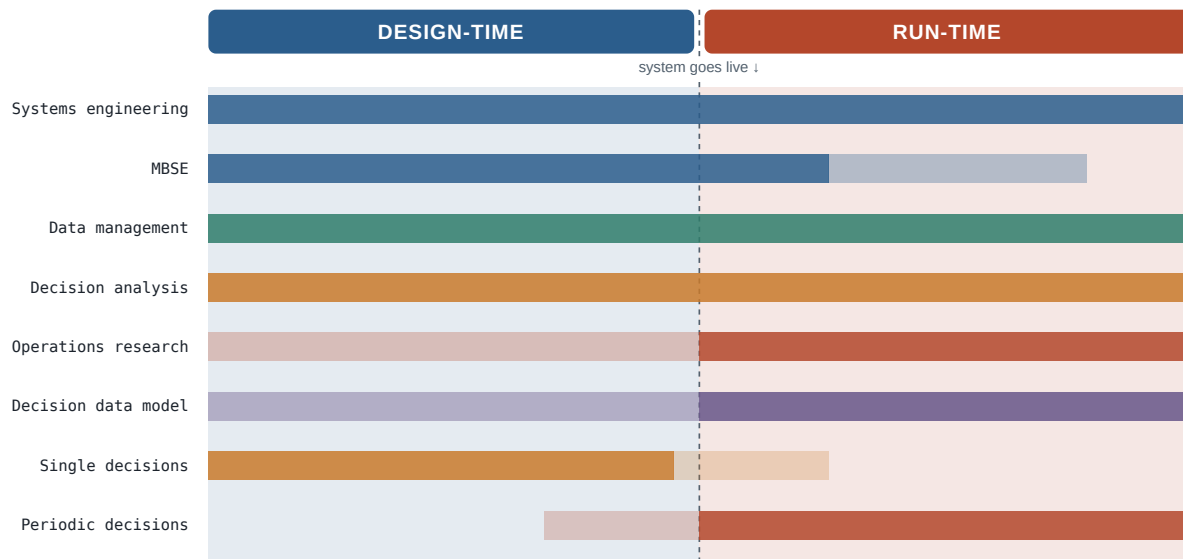


Figure 3 — where each discipline's work falls. The Operate & Sustain row added above is what finally closes the faded run-time segments on MBSE, operations research, the decision data model, and periodic decisions — see Part IV.

The return on investment is highest at the moment of change impact assessment — a component is redesigned, a requirement is tightened, an interface is redefined late in the program. Without the architecture, this requires manually identifying every dependent analysis, re-running it, reconciling results — weeks, and invariably incomplete. With the architecture, change impact assessment is a query: the SysML v2 API returns all elements referencing the changed component, the knowledge graph returns all dependent simulations and verification results, the RSM surrogate re-evaluates the affected subsystem in seconds.

Programs operating without the decision architecture exhibit characteristic pathologies: **analysis synchronization meetings** where discipline leads manually reconcile boundary conditions; **late-phase interface surprises** discovered during integration rather than model assembly; **design decisions attributed to engineering judgment** rather than traceable analysis; and **verification coverage gaps** discovered at audit rather than by query.

8. The Knowledge Graph: Making Traceability Structural

Document-based traceability — requirement traceability matrices, test reports, interface control documents — breaks under the pace of modern program change because documents cannot be automatically updated when the artifacts they reference change. The knowledge graph replaces the document fabric with a typed, directed graph where nodes are artifacts and edges are relationships.

Node Type	Source	Key Attributes	Connected To
ArchitectureElement	SysML v2 REST API	UUID, type, name, commit version, constraint expressions	RequirementSatisfaction, SimulationInterface, VerificationResult
SimulationConfiguration	FMU model descriptions + DoE records	Tool versions, parameter values, FMU versions, operating conditions	ArchitectureElement (interfaces), SurrogateModel, VerificationResult
SurrogateModel	RSM fitting records	Polynomial coefficients, DoE design, validation metrics, valid bounds	SimulationConfiguration (training data), ParetoFrontier
ParetoFrontier	NSGA-II / optimizer records	Objective values, design variable values, commit timestamp	SurrogateModel, ArchitectureElement (requirements), VerificationResult
VerificationResult	HiL test records + simulation results	Pass/fail, timestamp, operator, configuration version	ArchitectureElement (requirement), SimulationConfiguration, ParetoFrontier

With this schema, program decisions become queries. "Which Pareto frontier candidates have been HiL-verified against the current architecture commit?" is a three-hop graph traversal. "Which requirements have no passing verification result against any simulation configuration built from the current SysML v2 model version?" is a two-hop traversal with a NOT EXISTS filter. Milliseconds. Their document-based equivalents take days.

THE CIRCUIT BREAKER PATTERN FOR DECISION QUALITY

As AI assistants are increasingly deployed to support engineering workflows — recommending simulation configurations, interpreting Pareto frontiers, generating SysML model fragments — a failure mode emerges: AI-generated assertions that are plausible and confidently stated but inconsistent with the verified engineering record. The Circuit Breaker pattern grounds AI outputs against the knowledge graph before they reach engineers: before

an AI assistant makes a claim, it retrieves the relevant nodes and checks its assertion against retrieved ground truth. Discrepancies trigger a hold rather than silent propagation.

Mean Time Between Hallucinations (MTBH)

MTBH is a proposed metric for AI grounding effectiveness in engineering decision workflows: the rate at which AI-generated engineering assertions that were not validated against the knowledge graph later prove inconsistent with verified system specifications. Lower MTBH correlates with better grounding discipline and higher confidence in AI-assisted decision speed gains. *Part IV extends this flat, single-hop check into a recursive form for multi-domain, multi-agent environments.*

9. Competitive Advantage: What This Architecture Makes Possible

CAPABILITY	WITHOUT THE ARCHITECTURE	WITH THE ARCHITECTURE
Design space exploration	Evaluate 5–10 design alternatives per phase; choose by judgment	Evaluate 1,000+ alternatives via surrogate; choose from Pareto frontier
Change impact assessment	2–4 weeks of analysis re-running and reconciliation	Hours: graph query identifies stale analyses; surrogate re-evaluates immediately
Requirements verification	Manual comparison of simulation results against requirement documents	Automated: SysML v2 constraint expressions checked against simulation outputs
Multi-discipline trade-offs	Negotiated in review meetings; outcome depends on organizational dynamics	Characterized on a Pareto frontier; stakeholders choose with explicit trade-off awareness
Model-to-hardware correlation	Discovered at integration test; retroactive model corrections	HiL correlated against MBSE model; model updated proactively before integration
AI-assisted engineering	AI suggestions lack grounding; quality uncertain; verification burden on engineers	AI claims validated against knowledge graph before presentation; grounded acceleration

Programs operating with the decision architecture can make claims in bids and proposals that programs without it cannot credibly make: **schedule confidence** ("our design space was characterized by a validated surrogate across 847 design configurations before PDR"), **late-change resilience** ("we can identify all affected analyses within hours"), **verification**

completeness ("every Pareto frontier candidate carries a requirements traceability chain"), and **integration risk** ("model-to-hardware correlation data is in the program knowledge graph").

The Holonic Extension

Closing the run-time gap, wiring causal impact instead of reachability, and grounding multi-agent alignment

Parts I–III are a rigorous design-time decision accelerator that stops the moment a system goes live. Its knowledge graph is a single centralized semantic authority, not a federation. Its change-impact query is reachability, not causal magnitude. And nothing in it describes what a deployed system's operators, agents, or AI assistants should do once the design phase ends. This part closes those three gaps with one structural move: replace the centralized graph with a holarchy of domain-bounded ontologies, each locally deterministic, mutually governed through an audited meta-governance scaffold — the same move that answers a harder question raised alongside this architecture: how a multi-agent AI system stays capable and aligned at once.

GAP	V1	V2 MECHANISM
Run-time coverage	Architecture stops at Integration & Test	Operate & Sustain phase (§7) — digital-twin MBSE + periodic re-optimization, triggered by domain events
Relational structure (DDD)	Pull-only queries; no context map; no domain events	Domain events bridge design-time changes and run-time triggers; explicit trust-scoped boundaries between domains
Governance topology	One centralized semantic authority	Holarchy of domain-bounded ontologies (SHACL/SWRL), federated through a meta-governance scaffold
Impact assessment	Graph reachability ("connected to")	RSM β -coefficients as directional, weighted causal edges; STPA/STAMP generalized beyond AADL timing

10. From Centralized Model to Holarchy

§4.1's knowledge graph names five node types but treats them as peers in one flat schema, governed by one authority — the SysML v2 architecture model, described explicitly as the system's authoritative semantic contract, singular. That's a workable design-time pattern. It is

not a holonic one, and it doesn't survive contact with a multi-agent, multi-domain operating environment — exactly the environment the Operate & Sustain phase introduces.

The fix is structural, not additive. Each engineering domain — thermal, structures, controls, and however many more a program needs — becomes a **holon**: a bounded context that is simultaneously whole and part. Whole, because it owns a local ontology, expressed as SHACL shapes (what a valid claim in this domain looks like, structurally) and SWRL rules (what a valid claim in this domain implies, inferentially). Any agent operating inside that domain — human or AI — is constrained by those shapes and rules at the data layer; it cannot assert or act on a claim its local ontology rejects, regardless of its own internal objective. Part, because no domain talks to another domain directly. Every cross-domain request, and every audit of one, passes through a meta-governance scaffold holding cryptographic identity and a scoped trust boundary per domain, logged as a first-class graph node — the same pattern §8 already used for verification results, now applied to inter-domain traffic instead of hardware correlation.

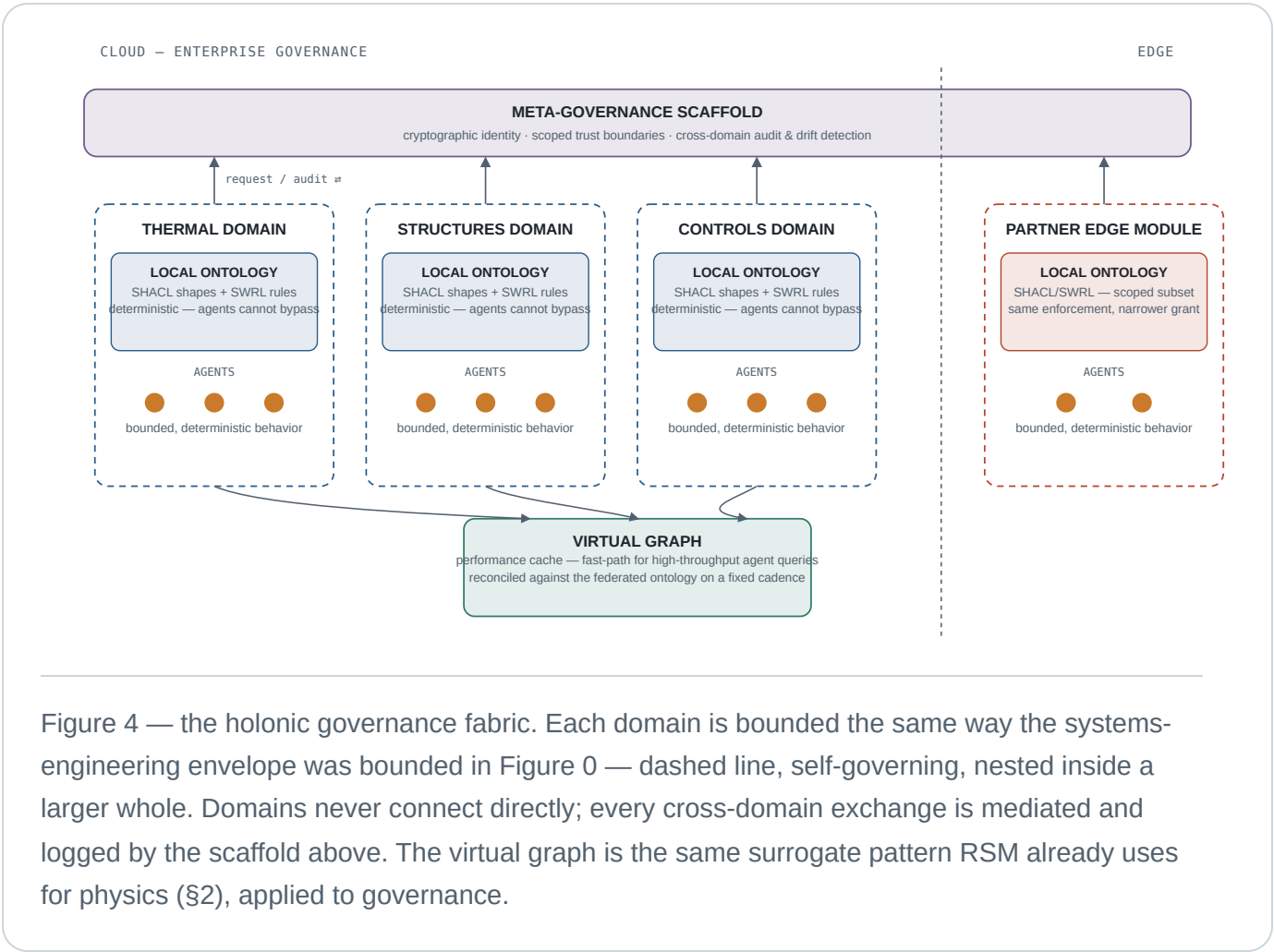


Figure 4 — the holonic governance fabric. Each domain is bounded the same way the systems-engineering envelope was bounded in Figure 0 — dashed line, self-governing, nested inside a larger whole. Domains never connect directly; every cross-domain exchange is mediated and logged by the scaffold above. The virtual graph is the same surrogate pattern RSM already uses for physics (§2), applied to governance.

This is also where the Circuit Breaker pattern (§8) has to change shape. A flat, single-hop grounding check is fine when a claim lives inside one domain. It isn't sufficient once a claim crosses domain boundaries — "raising this component's thermal limit is safe for the subsystem and the system" touches at least three holons. The v2 Circuit Breaker recurses: it validates the claim against the originating domain's ontology, then against the meta-governance scaffold's cross-domain rules, then against each domain the claim's consequences reach — and a violation at any level trips the breaker before the claim propagates further. That is a circuit breaker in the literal sense, not a single fuse.

11. Reachability Isn't Causality

§7's change-impact query is graph reachability: "return all elements that reference the changed component." That answers what's connected. It doesn't answer how much, through what mechanism, or whether the change is even permissible under the affected domain's own rules — three different questions that reachability collapses into one.

The RSM layer already computes the first of those without knowing it. The six coefficients in the fitted response surface (§2) are local sensitivities — β_1 is literally "how much does this objective move per unit change in this design variable." Wiring those coefficients onto the graph as directional, weighted edges — rather than leaving them inside the surrogate's regression output — turns "is this connected" into "by how much," with no new analysis required, only a different place to write down a number the architecture was already computing.

The second gap — mechanism and permissibility — is what the domain-local SWRL rules are for. A causal edge crossing from one domain into another is evaluated against the destination domain's rules before it registers as a valid impact, not merely logged as a reference. STPA/STAMP, which §4 uses only to certify AADL timing behavior, generalizes naturally here: its control-structure view of hazards as violations at a control boundary is exactly the shape a cross-holon causal edge needs to be checked against.

12. What This Resolves for Multi-Agent Alignment

The questions this section answers aren't a separate problem from the run-time and governance gaps above — they're the same problem, asked about AI systems instead of

engineering programs.

UNALIGNED AI

An unaligned model is one whose internal objective diverges from the intended one. The standard fix is model-level inspection: interpretability, red-teaming, watching for reward hacking after the fact. This architecture makes that problem less load-bearing: if an agent's internal objective diverges but the only actions available to it are the ones its local domain's SHACL/SWRL rules permit, misalignment inside the model can't produce a misaligned outcome in the system — the deterministic layer enforces intent, not the model's own self-report of its intent. Governance moves from "trust the model, then audit it" to "constrain the model's action space, then let it be as capable as it wants inside that space."

COMPONENTIZE VS. CAPABILITY

Strict compartments are inspectable but lose emergent reasoning; large unified models are capable but opaque. This architecture puts the trade-off at a different layer than the model: capability lives inside the domain — an agent reasons as flexibly as it needs to, so long as its outputs are checked against the local ontology before they act — while safety lives at the boundary between domains, enforced by the meta-governance scaffold, not by shrinking the models themselves.

THE NEURON ANALOGY

Layering many small models like neurons breaks down because rich real-time communication between them is expensive and lossy. This architecture makes the fast path cheap and the accurate path slow, deliberately — the same way RSM stands in for full MBSE simulation. The virtual graph is a validated cache of the federated ontology, reconciled on a fixed cadence; high-throughput agent interaction reads from it instead of traversing the full federated graph on every exchange.

SWARM OBSERVABILITY

Managing individual and collective agent behavior together comes down to simple, strict constraints locally and coordination protocols with observability collectively. Local SHACL/SWRL rules are the strict constraints on individual agents. The scaffold's audit log, as a queryable graph node type, is the observability layer: cross-domain traffic is inspectable the same way §8 already made verification results inspectable.

CLOUD / EDGE DEPLOYMENT

Private cloud for core governance, edge for partner-facing modules, maps directly onto the domain/edge-module distinction in Figure 4: the federated ontology and meta-governance scaffold stay in the enterprise's own cloud, while a partner-facing edge module gets a scoped, narrower subset of the same ontology, enforced by the same mechanism, with a smaller grant.

The engineering-decision problem and the AI-alignment problem are the same problem at different scales: how do you keep a system that's large, capable, and interconnected inspectable without cutting it into pieces too small to reason well. Centralize everything and you get a system nobody can audit. Compartmentalize everything and you get a system too dumb to be useful. The answer in both cases is the same shape — bounded local determinism, federated through an audited seam — not a bigger model, and not a smaller one.

13. Implementation Maturity, Extended

Programs at different starting points enter the maturity progression at different stages. The progression is cumulative — each level builds on the previous — and each level delivers measurable decision quality improvement even without the levels above it.

MATURITY LEVEL	CAPABILITY	IMMEDIATE DECISION BENEFIT	ENABLING TECHNOLOGY
Level 1: Formal interfaces	SysML v2 interface declarations with typed quantity kinds and units	Automated interface conformance checking; unit mismatch prevention	SysML v2 tool + REST API access
Level 2: Surrogate pipeline	DoE-sampled MBSE simulation + RSM surrogate per discipline	Design space exploration at 50–100× lower cost; sensitivity ranking	Modelica/Simulink + DoE software + Python/R RSM fitting
Level 3: Pareto integration	Multi-objective optimizer sweeping validated surrogates	Pareto-grounded trade-off conversations at design reviews; dominated designs eliminated	NSGA-II implementation (pymoo, MATLAB) + multi-surrogate interface
Level 4: Knowledge graph	Traceability graph connecting all artifacts through typed relationships	Change impact in hours; verification gap detection before audit; version-aware stale analysis	Neo4j or RDF store + SysML v2 API ingestion + FMU manifest parsing
Level 5: AI grounding	Circuit Breaker pattern; MTBH instrumentation	AI-assisted decision speed with verifiable quality; continuous grounding discipline improvement	LLM integration with knowledge graph query; MTBH monitoring
Level 6: Holonic federation (v2)	Domain holons + SHACL/SWRL + meta-governance scaffold + recursive Circuit Breaker	Cross-domain change impact with causal grounding, not reachability; AI-assisted operational decisions structurally aligned, not just monitored; Operate & Sustain phase closes the run-time gap	Federated graph store (domain-partitioned) + SHACL/SWRL engine + cryptographic-identity scaffold + virtual graph cache

The most important transitions are Level 1 to Level 2 — from formal interfaces to a working surrogate pipeline, where decision tempo actually changes — and Level 5 to Level 6, where governance stops being a single centralized authority and becomes a federation that survives contact with run-time, multi-agent operation.

§ Conclusion: The Decision Architecture as Engineering Practice

The methods examined here — MBSE simulation, Response Surface Methods, Pareto Optimality, and now a holonic knowledge federation — are individually well-established and individually valuable. Their value multiplies when used as one architecture, connected through formal interfaces, versioned repositories, and a traceability graph that survives

contact with run-time, multi-agent operation. For chief architects and systems engineers, the commitment this requires is not primarily a tooling commitment — the tools exist and are maturing. The commitment is an information architecture commitment: treat the connections between methods, and between domains, as first-class engineering artifacts requiring the same discipline, versioning, and governance as the models themselves.

Programs that make that commitment gain something that cannot be bought with additional headcount or compute time: the ability to answer consequential engineering questions in hours rather than weeks, with confidence that the answer is grounded in validated physics and traceable to formal requirements — and, increasingly, the ability to let AI agents accelerate that work without trading away the ability to inspect and trust it.

The binding constraint on engineering program performance — and on safe AI-agent performance — is not capability.

It is uninspectable interconnection.

The decision architecture collapses both without sacrificing rigor.

§ Appendix: Quick Reference

A. Method Selection Matrix

IF YOUR DECISION IS ABOUT...	LEAD WITH	SUPPORT WITH
Which system concept to pursue	Pareto (lightweight surrogates over concepts)	SysML v2 for requirements; RSM for objective functions
Sensitivity of performance to design parameters	RSM (DoE sampling + surface fitting)	MBSE simulation as the evaluation oracle
Optimal values for continuous design parameters	RSM + gradient-based optimizer	Pareto if multiple objectives; MBSE simulation for verification
Trade-offs between conflicting system objectives	Pareto (NSGA-II frontier)	RSM surrogates for efficiency; MBSE for frontier verification
Whether the design meets requirements at all conditions	MBSE co-simulation (full physics)	SysML v2 requirements traceability; HiL for hardware fidelity
Impact of a proposed design change	SysML v2 API query + RSM re-evaluation	Pareto re-sweep if objectives shift; causal edges for magnitude (v2)
Whether the real hardware matches the model	HiL correlation test	MBSE model update with correlated behavior
Whether an AI-assisted claim can be trusted	Circuit Breaker grounding check (v2: recursive across domains)	Knowledge graph query; MTBH instrumentation

B. Key Acronyms

ACRONYM	EXPANSION	SECTION
AADL	Architecture Analysis and Design Language (SAE AS5506)	§4
CCD	Central Composite Design	§2
DoE	Design of Experiments	§2
EMO	Evolutionary Multi-objective Optimization	§3
FMI / FMU	Functional Mockup Interface / Unit	§5
HiL / SiL / PiL / MiL	Hardware / Software / Processor / Model-in-the-Loop	§6
MBSE	Model-Based Systems Engineering	Throughout
MTBH	Mean Time Between Hallucinations	§8
NSGA-II / III	Non-dominated Sorting Genetic Algorithm II / III	§3
Pareto	Pareto Optimality Analysis	§3
RSM	Response Surface Methods	§2
SysML	Systems Modeling Language (OMG standard)	§4
Holon	A bounded domain that is simultaneously whole (self-governing) and part (subordinate to the scaffold)	§10
SHACL	Shapes Constraint Language — structural validation of graph claims	§10
SWRL	Semantic Web Rule Language — inferential/causal rules on graph claims	§10
Meta-governance scaffold	The inter-domain layer holding identity, trust boundaries, and audit	§10
Virtual graph	Performance cache of the federated ontology, reconciled on a cadence	§10
Domain event	A published change letting design-time and run-time contexts react without polling	§10